

Data Warehousing and OLAP Technology: An Overview

- A multi-dimensional data model
- Data warehouse design

OLAP的概念

- 最早是由关系数据库之父E.F.Codd于1993年提出的。
- 当时，Codd认为联机事务处理已经不能满足终端用户对数据库数据查询和分析的需要，SQL对大型数据库进行的简单查询也不能满足用户分析的需求，用户的决策分析需要对关系数据库进行大量计算才能得到结果，而简单的查询并不能满足决策者提出的要求。

OLAP的定义

- 不同的专家和组织对OLAP赋予了不同的含义。
- OLAP委员会给出的一个定义：
 - OLAP是使分析人员、管理人员或执行人员能够从多种角度对从原始数据中转化出来的、能够真正为用户所理解的、并真实反映企业多维特性的信息进行快速、一致、交互地存取，从而获得对数据的更深入了解的一类软件技术。

OLAP的特点

- **OLAP**是供管理人员使用的一种软件，物理上要能对用户提出的各类复杂查询快速响应，逻辑上要具备多维建模的能力，主要特点有：
 - **快速性**：用户对**OLAP**的快速反应能力有很高的要求，系统应能在很短的时间内对用户的大部分分析要求做出反应。
 - **可分析性**：**OLAP**系统应能处理与应用有关的各种逻辑分析和统计分析。

OLAP的特点

- 多维性：系统必须提供对数据的多维视图和分析，包括对维层次和多重维层次的完全支持。
- 及时性：不论数据量有多大，也不管数据存储在哪里，**OLAP**系统应能及时获得信息，并且管理海量信息。

OLAP与OLTP的区别

- OLAP与OLTP是两类不同的应用。
 - OLTP是对数据库数据的联机查询和增删改操作，它以数据库为基础
 - OLAP更适合以数据仓库为基础的数据分析处理
- OLAP中历史的、导出的及经综合提炼的数据均来自OLTP所依赖的底层数据库。

OLAP与OLTP的区别

- OLAP数据较之OLTP数据要多一步数据多维化或预综合处理的操作。
 - 例如，对一些统计数据，首先进行预综合处理，建立不同粒度，不同级别的统计数据，从而使其能满足快速数据分析和查询的要求。

OLAP与OLTP的区别

- 除了数据及处理上的不同之外，**OLAP**的前端产品的界面风格及数据访问方式也与**OLTP**不同
 - **OLAP**大多采用非数据处理专业人员容易理解的方式（如多维报表、统计图形），查询及数据显示直观灵活，用户可以方便地进行逐层细化及切片、切块、数据旋转等操作
 - **OLTP**大多使用操作人员常用的固定表格，查询及数据显示也比较固定、规范。

OLAP的核心技术

- 多维数据模型
- 多维分析操作
- 多维查询及展示
- 数据方体技术

多维数据模型

- 数据分析时用户的数据视图，是面向分析的数据模型，用于给分析人员提供多种观察的视角和面向分析的操作。
- 多维数据模型是数据仓库和联机分析处理技术的基础。
- 多维数据模型的数据结构可以用这样的多维数组来表示：(维1, 维2, ..., 维 n ; 度量1, 度量2, ..., 度量 m),

多维数据模型

- 直观的讲，多维数据模型是一个多维空间，维表示用户的观察对象，多维空间中的点表示度量的值。
OLAP采用了多维数据模型。

多维数据模型

- 与关系数据模型不同，多维数据模型是随着OLAP产品的流行而出现的，因此，缺乏坚实的理论基础，目前还没有统一的多维数据模型。
- 理论界从不同的方面提出了一些模型，主要有：
 - 用关系模型表示多维数据模型
 - 把多维数据模型形式化为多维空间

多维数据模型

- 多维数据模型的核心概念：
 - 维
 - 维成员
 - 维层
 - 维层次
 - 度量

维

- 维是人们观察数据的特定角度，是某个事物的属性。例如，在分析产品销售数据时，涉及到时间、产品、地区。
 - 因为企业需要从时间的角度来观察产品的销售，观察随着时间推移而产生的变化情况，查看在某个时间或某个时间段产品的销售量，所以时间就是一个维（时间维）。
 - 企业也时常关心自己的产品在不同的地区的销售分布情况，这时他是从地理分布的角度来观察产品的销售，所以地理分布也是一个维（地区维）。
 - 当然，更关心某个具体的产品的销售情况，产品也是一个维。
- 维是商业活动中的一个基本要素。每个维都有一个唯一的名字，如时间维、地区维、产品维等。

维成员 (Member)

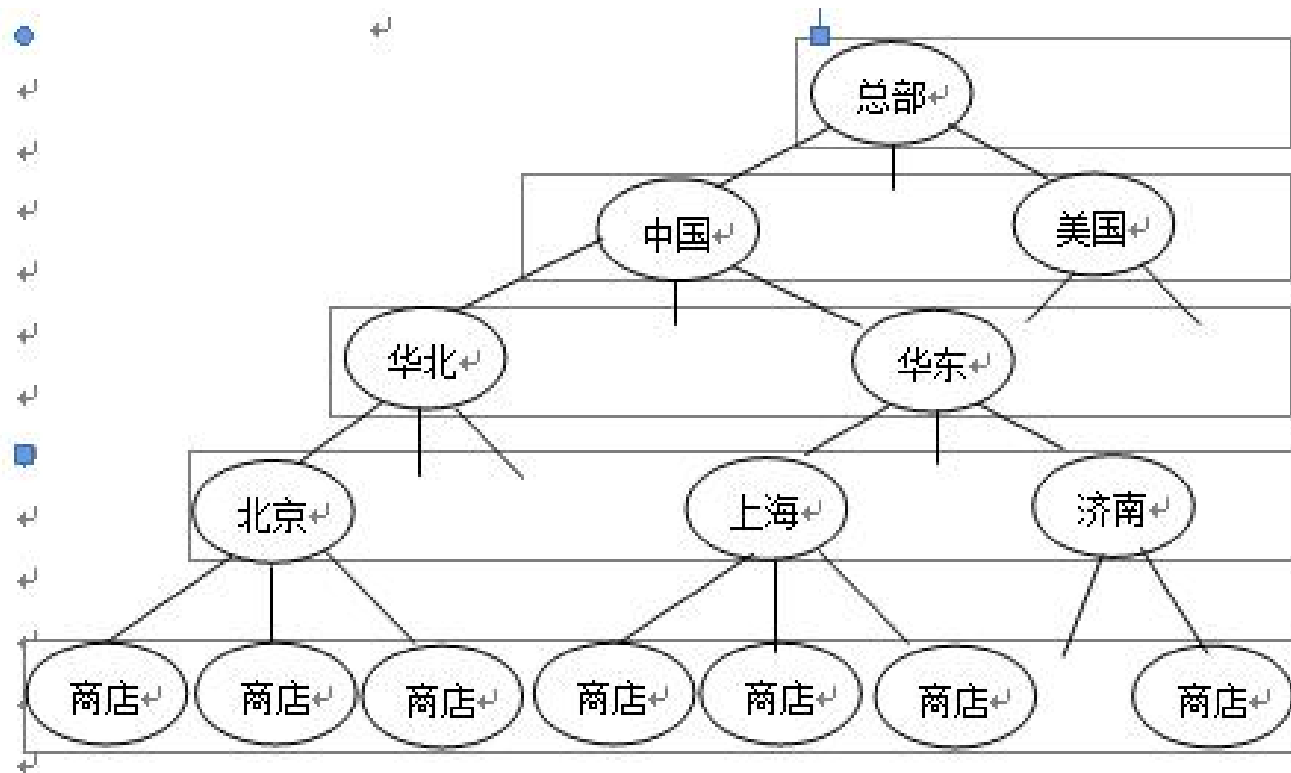
- 维由一些维成员构成。维的一个取值称为维的一个成员，每个成员有一个名字，还可以有若干的属性来描述成员的特征。
- 如果一个维是多层的，那么该维的维成员是在不同维层上的取值的组合。
- 例如，假设时间维具有日、月、年三个层。则分别在日、月、年上各取一个值组合起来，就得到了时间维的一个维成员，如 " 2008年8月8日 "。一个维成员并不一定在每个维层上都要取值，例如， " 2008年5月 "、 " 10月1日 "、 " 2008年 " 等都是时间维的维成员。

维层 (Level)

- 人们观察数据时，除了要从某一个角度去观察外，还需要从不同的细节程度去观察。称这些不同的细节程度为不同的维层。
- 例如，对于时间维，日、月、季度、年是不同的细节。需要知道每天的商品销售量，也需要知道每月、每季度、每年的商品销售量。那么日、月、季度、年等就是时间维的维层
- 同样，商店、城市、地区、国家，总部等构成了地区维的多个维层
- 维层描述现实中的细节程度、抽象级别，每个层都有一个名字，层之间一般存在由抽象级别决定的关系

维层成员

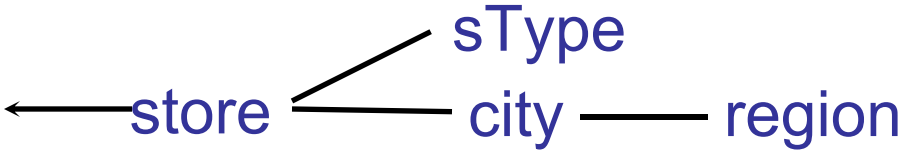
- 层除了名字外，还有成员。
- 例如，地区维中有商店、城市、地区、国家，总部五个层。
 - 在“城市”层，有成员“北京”、“上海”、“济南”、“芝加哥”、“洛杉矶”等。
 - 在“国家”层，有“中国”、“美国”等。
- 因此，维层实际上是对维成员的一种组织分类方法



维层次 (Hierarchy)

- 维层是一种分类方法。在一个维中可以有多种分类方法，每种分类方法叫做一个层次.例如：
 - 时间维，除了可以按年、季度、月、日分层外，还可以按年、星期、日分层，因此有两个层次。
 - 对于产品，客户分析员喜欢按产品、类型、种类来分析，股票分析员喜欢按产品、制造商、母公司来分析。为了满足分析的需要，产品维需要设置两个层次。

维层次



store	<u>storeId</u>	cityId	tId	mgr
	s5	sfo	t1	joe
	s7	sfo	t2	fred
	s9	la	t1	nancy

sType	<u>tId</u>	size	location
	t1	small	downtown
	t2	large	suburbs

city	<u>cityId</u>	pop	regId
	sfo	1M	north
	la	5M	south

region	<u>regId</u>	name
	north	cold region
	south	warm region

维属性

- 维属性说明维成员所具有的特征。
- 维属性可以在每个维成员上定义，也可以在维层上定义。
- 在维层上定义属性意味着维层中的每个维成员都具有该属性。例如：
 - 可以在地区维的商店这一个层上定义“负责人”、“开业时间”、“商店类型”等属性
 - 也可以在城市这个层上定义“人口”、“面积”等属性。

度量

- 要分析的目标或对象
- 常见的度量：
 - 销售量
 - 供应量
 - 利润
- 度量一般有名字、数据类型、单位、计算公式等属性

度量

- 度量有输入度量和导出度量之分,前者的值从业务处理活动中获取,后者需要经过计算得到。例如
 - 销售量是输入度量
 - 利润是导出度量。
- 度量还可以分为可累计型和不可累计型
 - 不可累计型的度量不能沿时间维做聚集运算
 - 例如,库存量就属于不可累计型度量,不能把一年12个月的库存量累加起来作为全年的库存量。

维表

- 维表描述一个维所需要的全部信息。属性有三种：主码、层属性、特征属性
 - 主码唯一地标识一行，每个最底层的成员要在维表中有一行，同其它的属性一起表示成员关系图中从最底层成员到最高层成员的一条路径
 - 层属性表示一个层的名称，其值是层中成员的集合，有一个层就要有一个层属性
 - 特征属性描述了成员的特征

维表是一个反规范化的表，通过函数依赖表示层之间的抽象程度，表达层次。

维表

商店ID	商店名称	商店负责人	经营面积	城市	城市人口	地区	国家
1	海淀1店	张明	100	北京	1000	华北	中国
2	海淀2店	李海林	86	北京	1000	华北	中国
...							
685	历下1店	王兆文	90	济南	300	华东	中国

维表

- 商店**ID**是主码，它唯一地标识一个商店。
- 商店名称、城市、地区、国家是层属性，在这些属性上的投影是层中的成员集合，例如，城市列中是所有城市成员。
- 商店负责人、经营面积、城市人口是特征属性。

维表

- 显然，维表中存在大量的冗余数据
- 如果对维表进行规范化处理，那么表示一个维就需要多个关系表，星型模型也改变成雪片模型
 - 对上例的维表进行规范化处理可以得到四个关系表
 - (商店ID, 商店名称, 经营面积, 城市ID)
 - (城市ID, 城市, 城市人口, 地区ID)
 - (地区ID, 地区, 国家ID)
 - (国家ID, 国家)

维表

- 规范化后消除了数据冗余，但是，对维的操作要涉及很多连接运算，计算代价很高
- 对**OLAP**来说，响应速度至关重要，因此，一般不对维表进行规范化处理，允许冗余，以空间换时间

维表

- 与事实表相比，维表中的记录数较少。一般，少的可能只有几百行，多的会达到百万级。
- 维表中存放的是描述信息，大多为文字资料。
- 维表中的信息可以修改。

事实表

- 事实表的属性分为两类：主码和度量。
 - 主码是一个复合码，每个分量是维表的主码。
 - 多维数据模型中的度量都作为一个属性出现在事实表中，一般都是数值型的量。

事实表

时间ID	商店ID	产品ID	销售量
1	1	1	300
1	2	1	450
1	3	1	360
1	1	2	400
1	2	3	130
2	3	2	200

事实表

- 前三列分别是时间维、商店维和产品维的关键字，最后一列是度量。
- 表中的每条记录表示一条事实，即在某时、某地，某种产品的销售量。
- 事实表中的量都是数值型的量，特别的，时间维也是数字，它是对时间的一个编号，即使跨越十年的时间，也不过**3600**多天，两个字节就足够了。

事实表

- 事实表用数字作为维关键字，可以加快连接操作。

因此，星型模型把大量的描述性的文字信息存放在维表，而在事实表中只存放数字型的信息，可以大大减少存储量，加快处理速度。

事实表

- 事实上，事实表是十分庞大的，记录数多，度量列多
 - 想象一下一个大型连锁店十年来在上百个商店，对上万种商品的销售记录数
- 另外，在实际的分析过程中，需要用到很多的输入度量和导出度量，事实表中的度量列也相当的多。

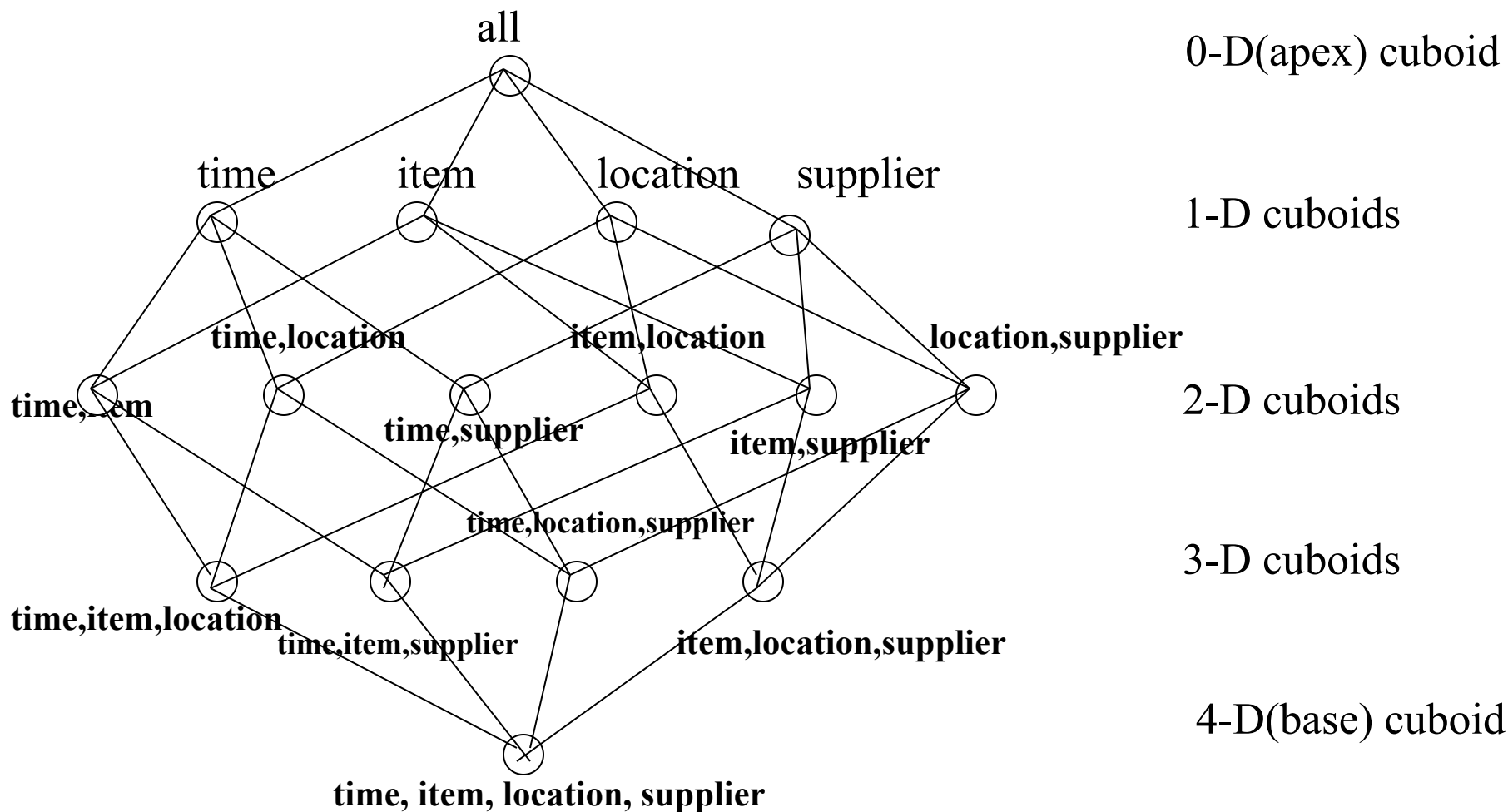
数据方体

- 多维数据模型构成的多维数据空间称作数据方体（**Data Cube**，简记为**Cube**），也称为数据立方体，超级立方体，多维超方体等
- 一个数据方体由多个维和度量组成
- 尽管最直观的数据方体是**3**维的结构，实际上它可以是**n**维的
- 虽然从逻辑上看数据方体是用多维方式表达的，但实际的物理存储方式未必是多维存储方式，可以用任意的方式存储。

From Tables to Data Cubes

- A **data cube**, such as **sales**, allows data to be modeled and viewed in multiple dimensions
 - **Dimension tables**, such as **item** (**item_name**, **brand**, **type**), or **time**(**day**, **week**, **month**, **quarter**, **year**)
 - **Fact table** contains measures (such as **dollars_sold**) and keys to each of the related dimension tables
- In data warehousing literature, an n-D base cube is called a **base cuboid**. The top most 0-D cuboid, which holds the highest-level of summarization, is called the **apex cuboid**. The lattice of cuboids forms a **data cube**.

Cube: A Lattice of Cuboids



数据单元

- 数据方体以维为数轴。
- 在一个数据方体中，当每个维上都确定了一个维成员时，就会唯一地确定多维空间中的一个点。我们把这个点称作一个数据单元（Cell）。

Conceptual Modeling of Data Warehouses

- Modeling data warehouses: dimensions & measures
 - Star schema: A fact table in the middle connected to a set of dimension tables
 - Snowflake schema: A refinement of star schema where some dimensional hierarchy is **normalized** into a set of smaller dimension tables, forming a shape similar to snowflake
 - Fact constellations: Multiple fact tables share dimension tables, viewed as a collection of stars, therefore called **galaxy schema** or fact constellation

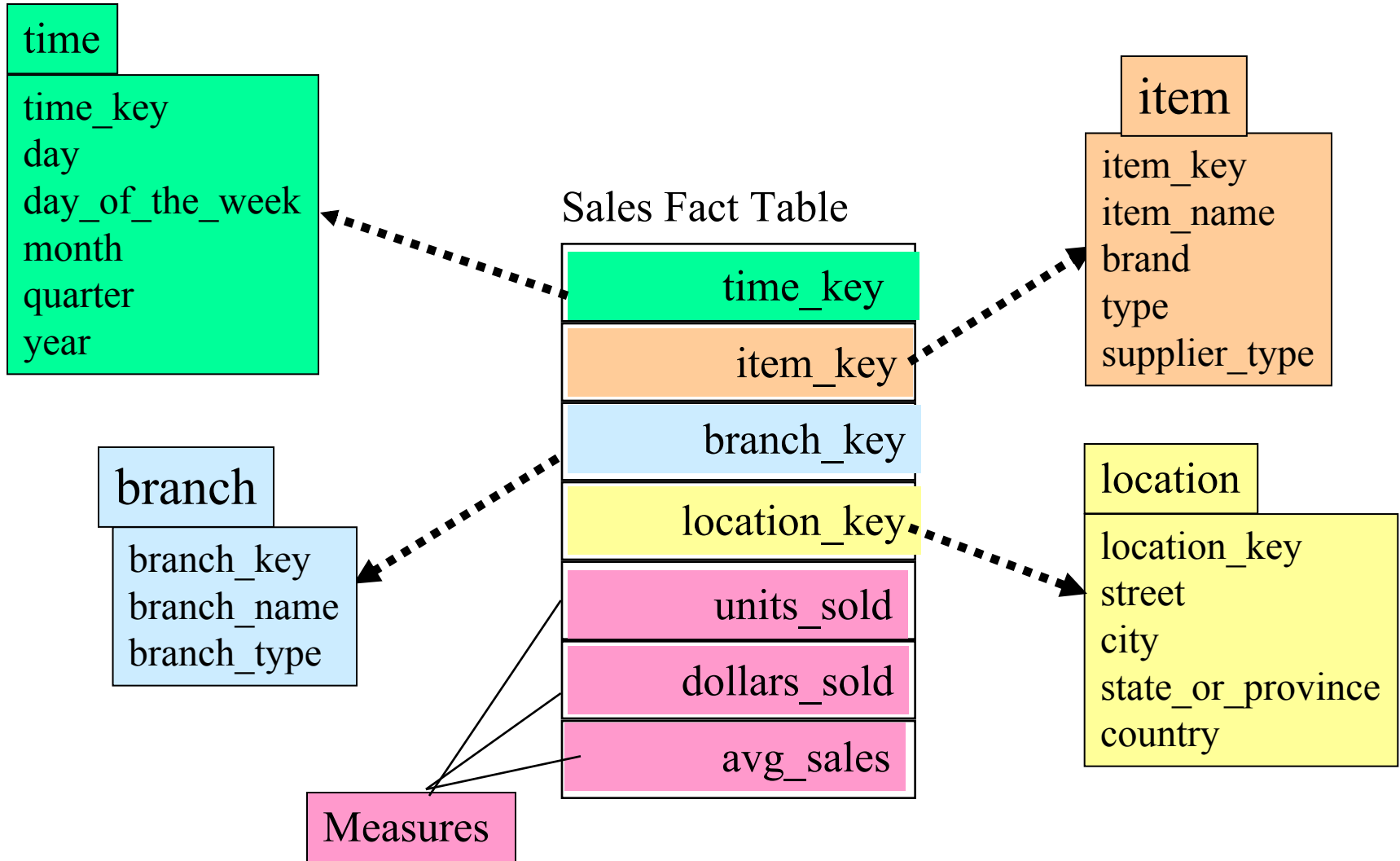
常见的多维数据模型

- 星型模型
- 雪片模型
- 事实群模型

星型模型

- 是多维数据模型的基本结构，通常由一个很大的中心表和一组较小的表组成
- 大的中心表通常被称为事实 (**fact**) 表。事实表用来存储事实的度量值及各个维的码值
- 与事实表相连接的周围的那组小的表通常被称为维表。维表用来保存维的信息，即每个维成员的信息，包括维的属性信息和维的层次信息等
- 事实表通过所存储的每个维表的码值和每一个维表联系在一起

星型模型



星型模型

- 星型模型不支持维的层结构。每个维只有一个维表，实现时将所有的维层属性存放在这一个表中，没有进行规范化。
- 因为要表示所有的层，每个层有自己的属性，所以有很多冗余。

星型模型

- 当不同的维层有相同的属性时，只能用换名的方法，但这会影响到一些查询。
- 例如商店->城市->地区->国家->总部层次确定了商店的隶属关系，如果在每一层上都有一个 **manager** 属性，查询 **manager=“Jones Smith”** 的结果应该是所有的叫 **“Jones Smith”** 的经理，而不论他是哪一级的经理。

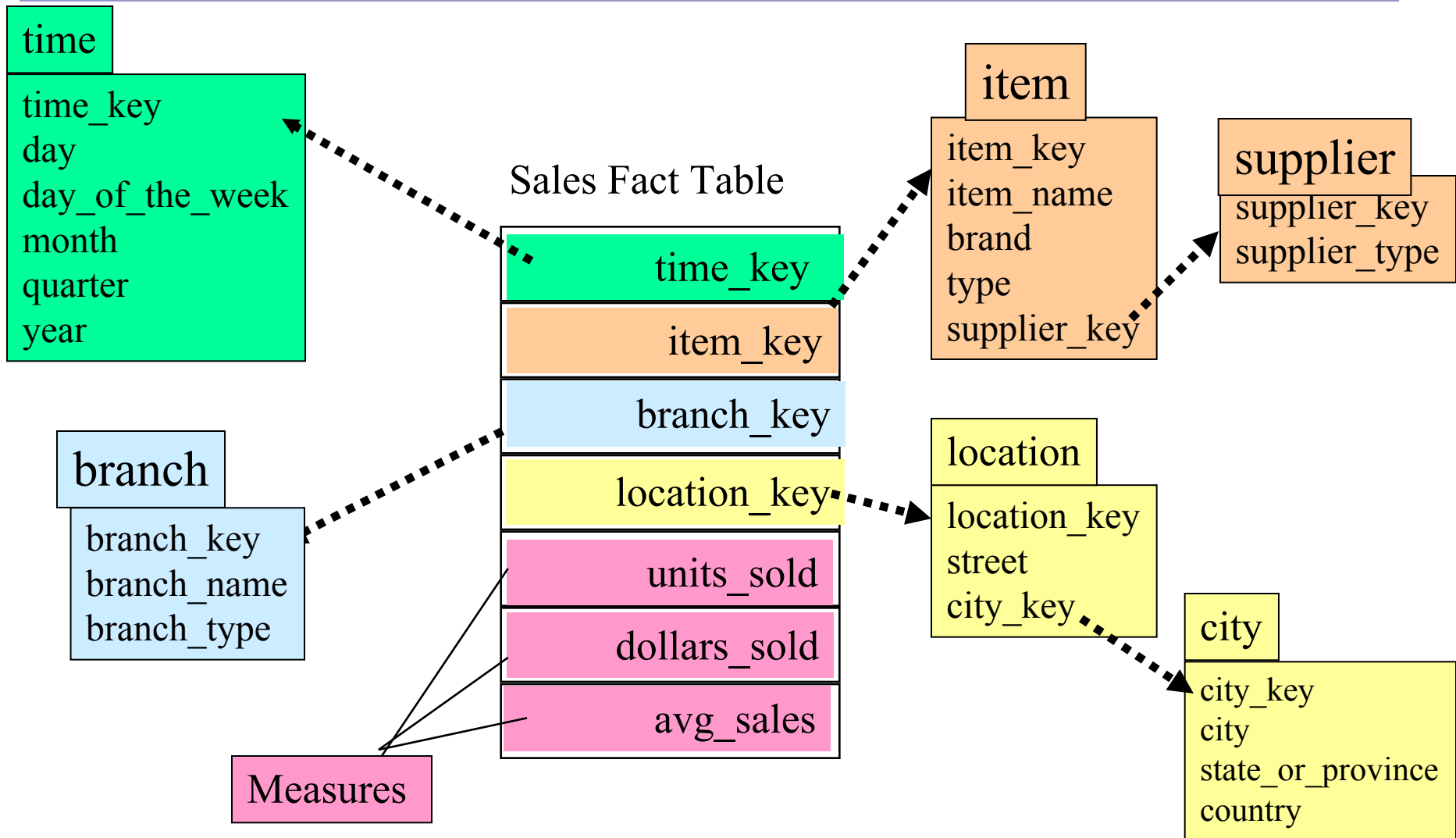
星型模型

- 对于维内层次比较复杂的维，为了避免冗余数据占用过大的空间，同时也为了支持针对不同层上的相同属性的查询，可以用多张表来描述一个复杂维。
- 比如，产品维可以进一步划分为商标表、类型表等，这样在“星”的角上又出现了分支，这种变种的星型模型被称之为“雪片模型”。

雪片模型

- 雪片模型，也称雪花模型，是对维表进行规范化后形成的

雪片模型



雪片模型

- 星型模型和雪片模型的主要区别在于雪片模型中的维表是规范化的，这样的维表不但易于维护而且节省存储空间。
- 例如，在商店->城市->地区->国家->总部这个维层次中，如果在每一层上都有一个**manager**属性的话，我们只需要在相应的表上存储当前层的**manager**属性值就可以了，而不需要在同一个表里存储所有层上的**manager**属性值。

雪片模型

- 但是，雪片模型的这种结构在执行查询时需要进行较多的连接操作，可能影响系统的性能。
- 例如，假如想查询某个地区下属的所有商店的 **manager** 时，需要将地区表、城市表和商店表进行连接操作。

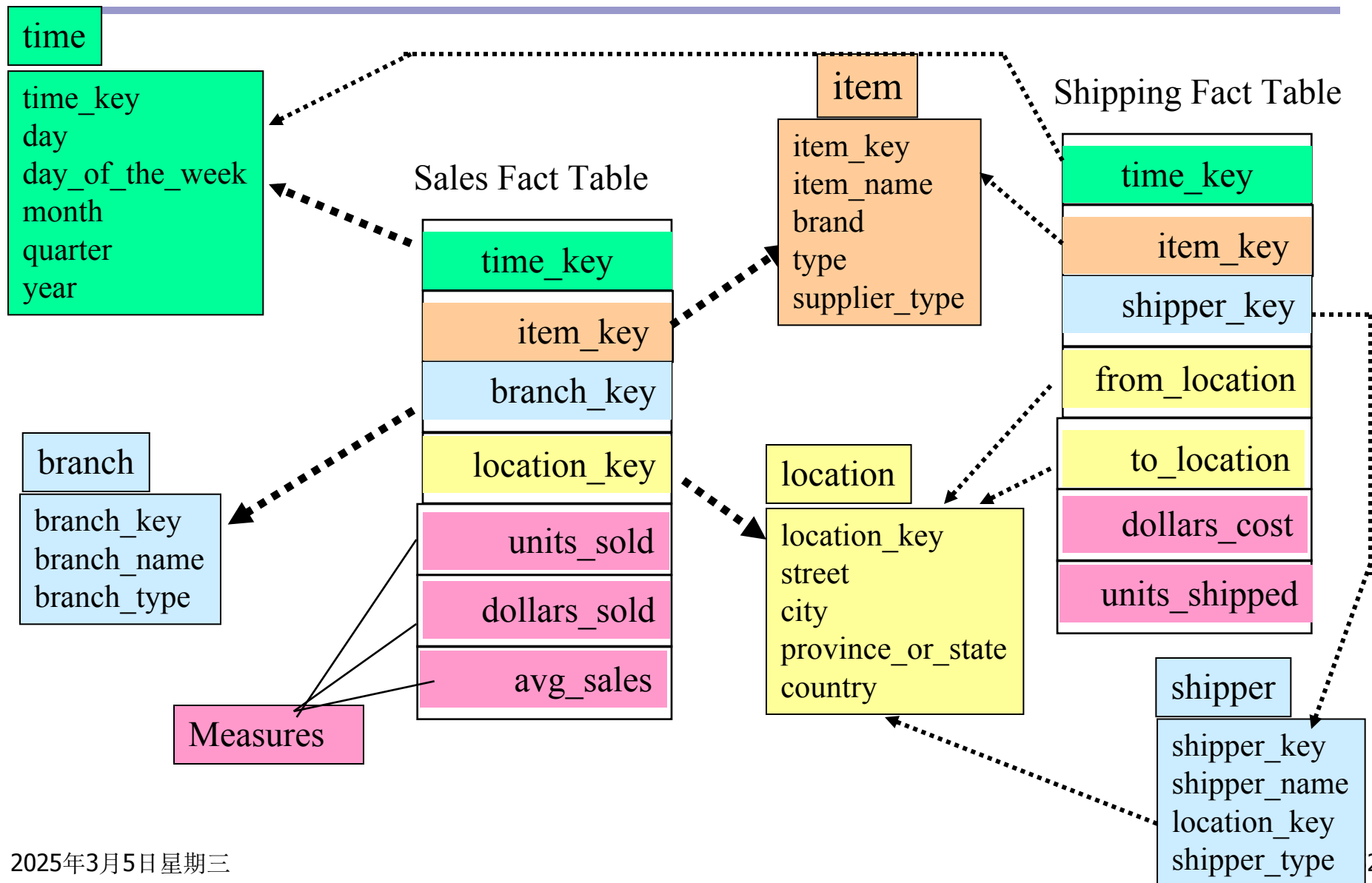
雪片模型

- 折衷的解决方式是将星型模型和雪片模型结合起来使用。
 - 对大的维表进行规范化以节省存储空间
 - 对小的维表仍然采用星型模型中的不规范形式，以避免由于多表连接引起的性能衰减
- 雪片模型更好地体现了维的层结构，对于数据库建模和设计人员来说，它非常容易理解和进行维分析。但对于一般用户来说，与星型模型相比，它相对比较复杂。

事实群模型

- 在某些复杂的应用中，可能需要多个事实表来共享维表，这种模型类似于星型模型的集合，被称为星系模型或事实群模型

事实群模型



多维分析的基础：聚集

- 聚集是对细节数据进行综合的过程，是多维分析的基础。
- 最常用的5种聚集函数是：求和（**sum**），计数（**count**），求最大值（**max**），求最小值（**min**），求平均（**average**）。
- 聚集函数还有求中间值（**median**）、排序（**rank**）等

数据方体中的三类聚集函数

- 1) 分布型 (distributive) 聚集函数
- 2) 代数型 (algebraic) 聚集函数
- 3) 整体型 (holistic) 聚集函数

分布型 (distributive) 聚集函数

- 将数据分成 n 份，对其中的每一份应用该函数，可以得到 n 个聚集值，对这 n 个聚集值进行计算得到的结果如果和整个数据（不划分）应用该函数得出的结果一致的话，则说该函数是分布型的
- 例如，使用**count**函数进行数据方体聚集计算时，可以先将数据划分为若干份，对每份应用**count**函数后再将所有的结果值相加得到最终结果
- 同理，**sum**、**min**、**max**也是分布型聚集函数。分布型聚集函数具有可以累计的特性。

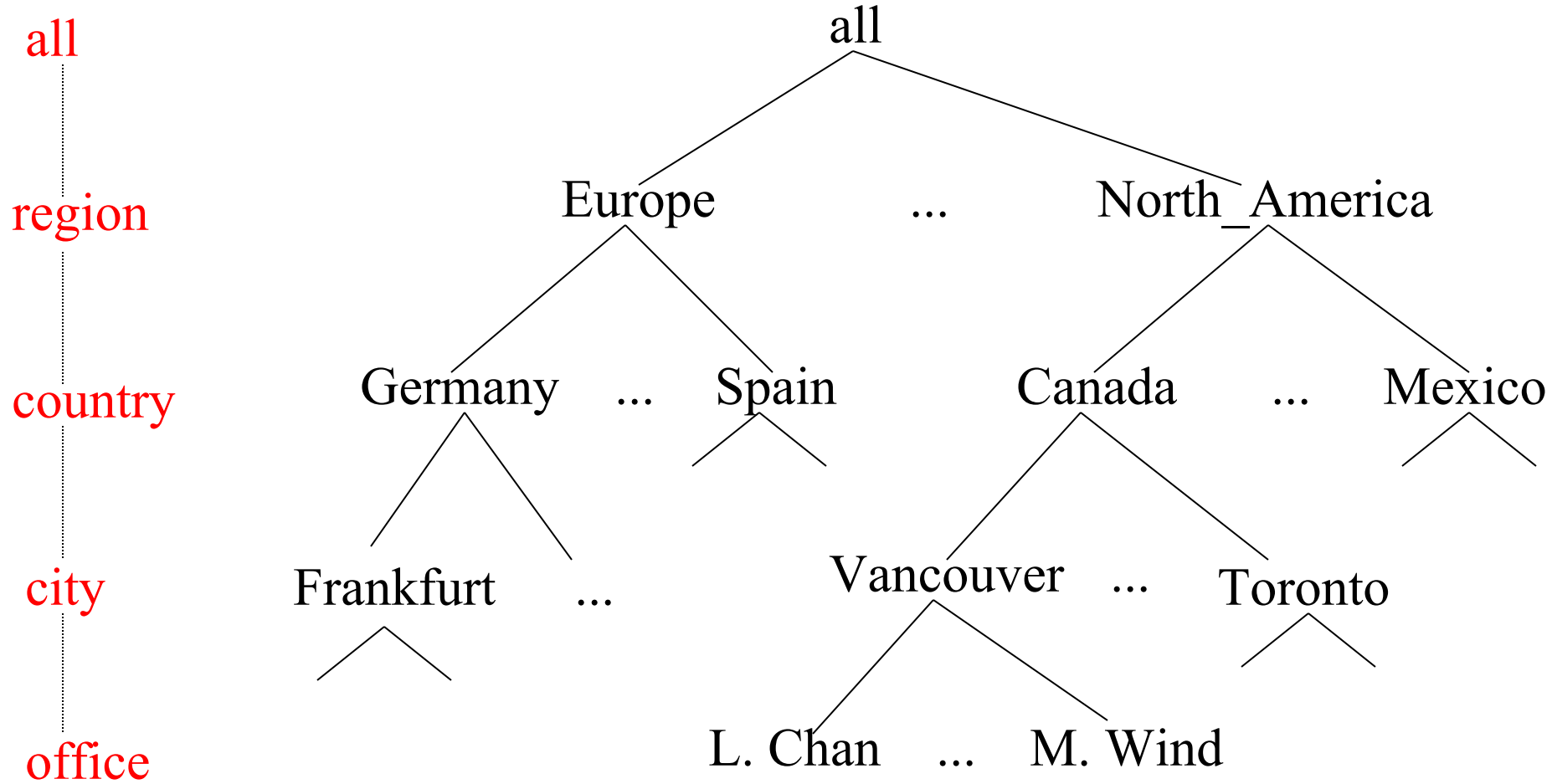
代数型 (algebraic) 聚集函数

- 如果一个函数可以由若干个分布型函数进行代数运算得出，则说该函数是代数型的
- 例如，**avg**可以通过**sum/count**得出，而其中的**sum**和**count**都是分布型函数，因此我们说**avg**是代数型函数
- 类似的，求最小的N个数所用的**min_N**函数，求最大的N个数所用的函数**max_N**，求标准方差的函数**standard_deviation**等都是代数型函数。代数型函数的典型特征是可以转化为分布型函数。

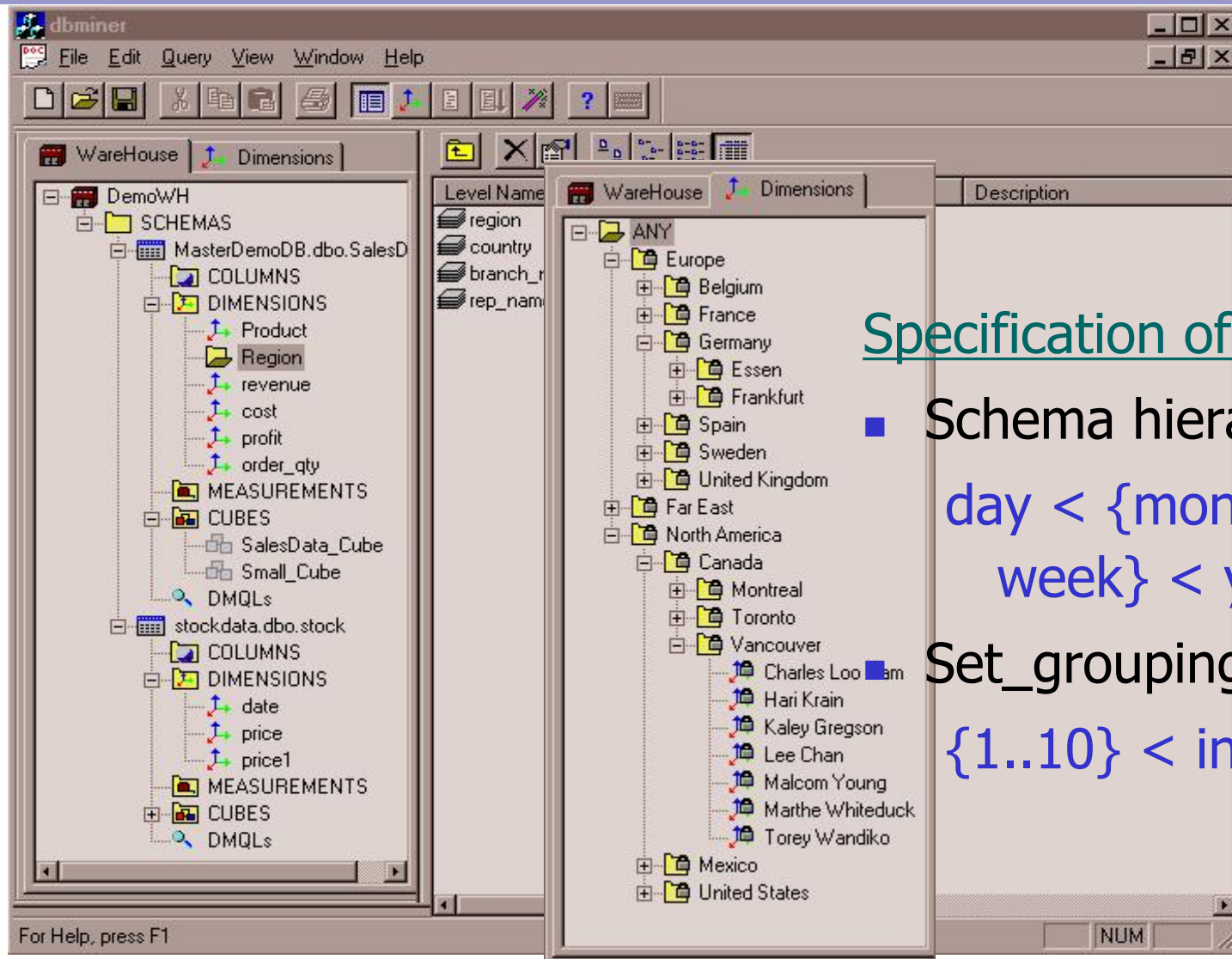
整体型（holistic）聚集函数

- 如果一个函数不能由其他的函数进行代数运算得出，则说该函数是整体型的
- 例如，**median**，**rank**等
- 这类函数在计算的时候需要同时用到所有的数据，需要消耗较多的资源，具有较大的计算代价

A Concept Hierarchy: Dimension (location)



View of Warehouses and Hierarchies



Specification of hierarchies

■ Schema hierarchy

day < {month < quarter;
week} < year

Set_grouping hierarchy

{1..10} < inexpensive

多维模型

Fact table view:

sale	prodId	storeId	amt
	p1	c1	12
	p2	c1	11
	p1	c3	50
	p2	c2	8



Multi-dimensional cube:

	c1	c2	c3
p1	12		50
p2	11	8	

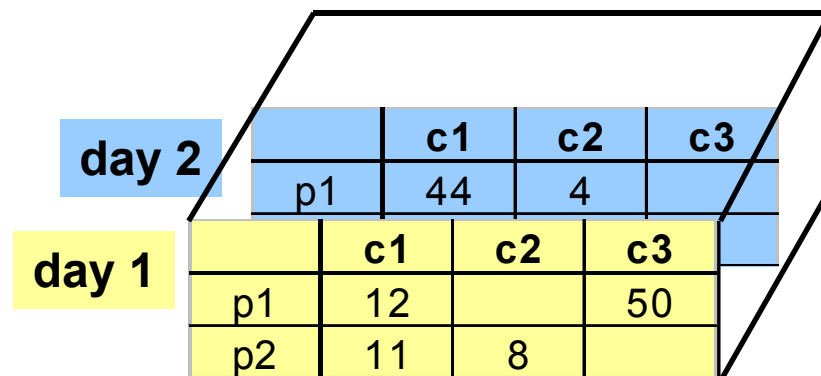
dimensions = 2

3-D Cube

Fact table view:

sale	prodId	storeId	date	amt
	p1	c1	1	12
	p2	c1	1	11
	p1	c3	1	50
	p2	c2	1	8
	p1	c1	2	44
	p1	c2	2	4

Multi-dimensional cube:

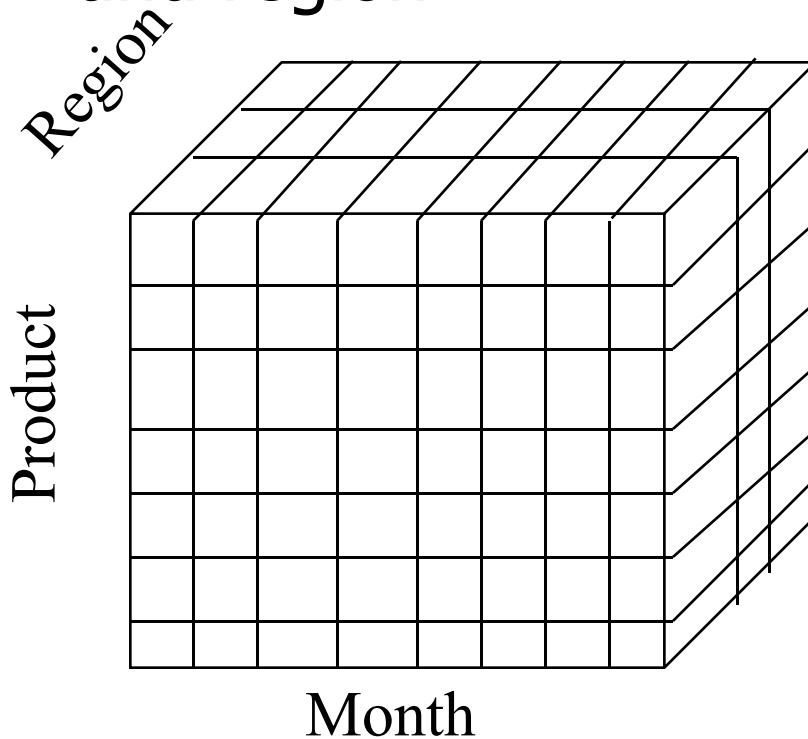


day 2			
	c1	c2	c3
p1	44	4	
day 1			
	c1	c2	c3
p1	12		50
p2	11	8	

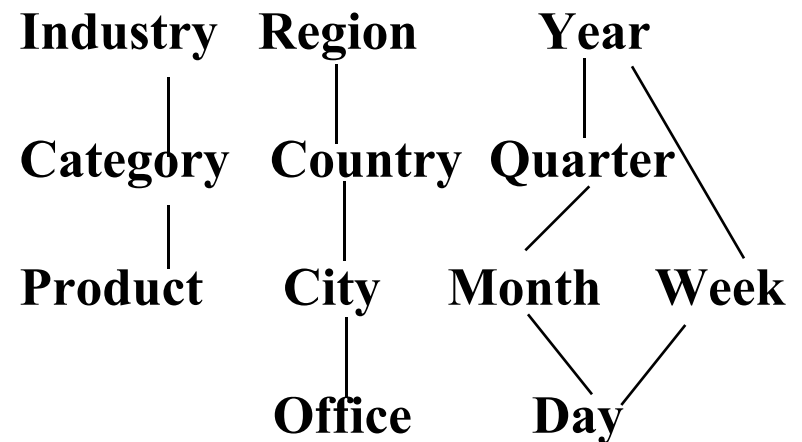
dimensions = 3

一个例子

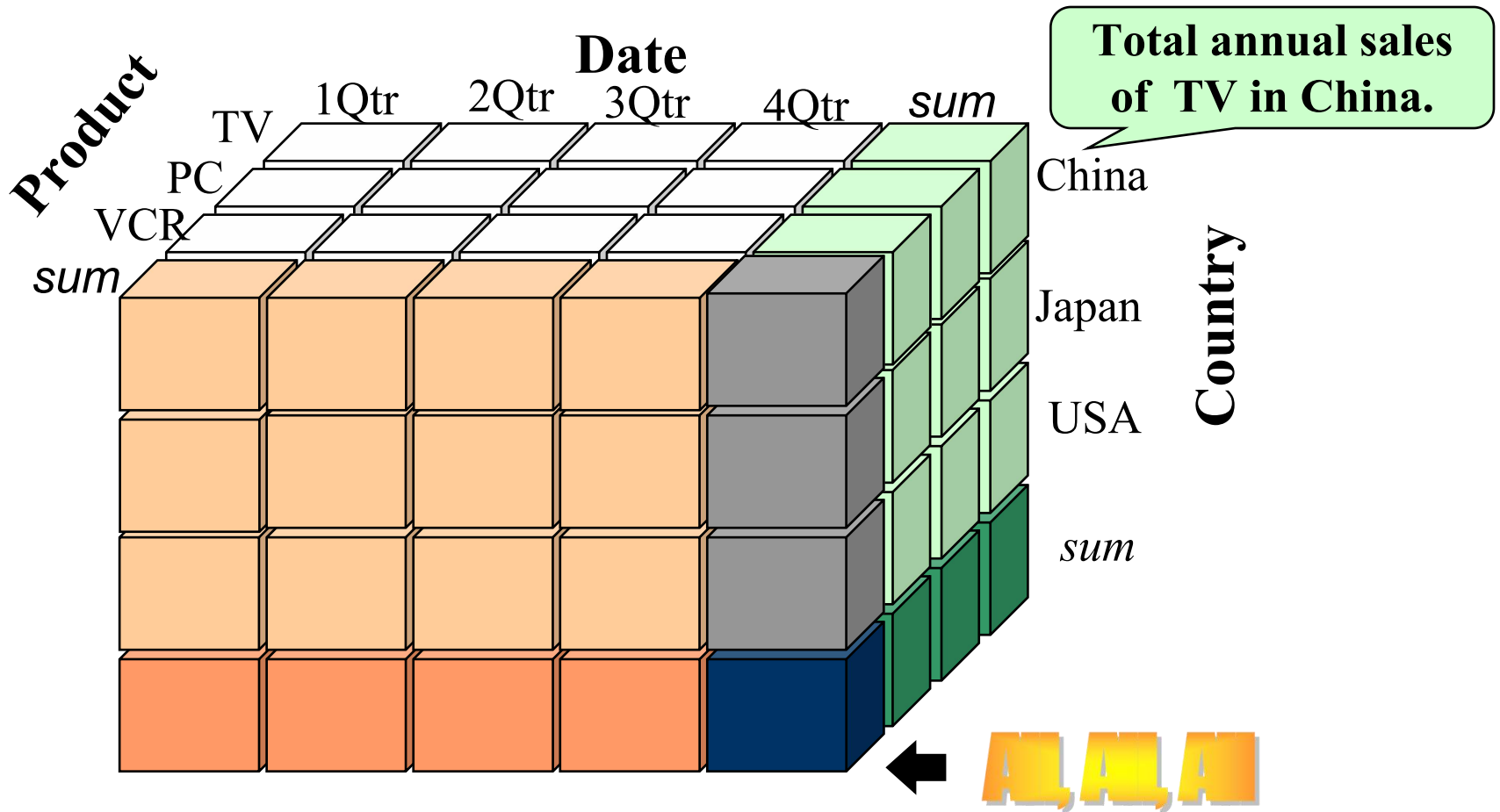
- Sales volume as a function of product, month, and region



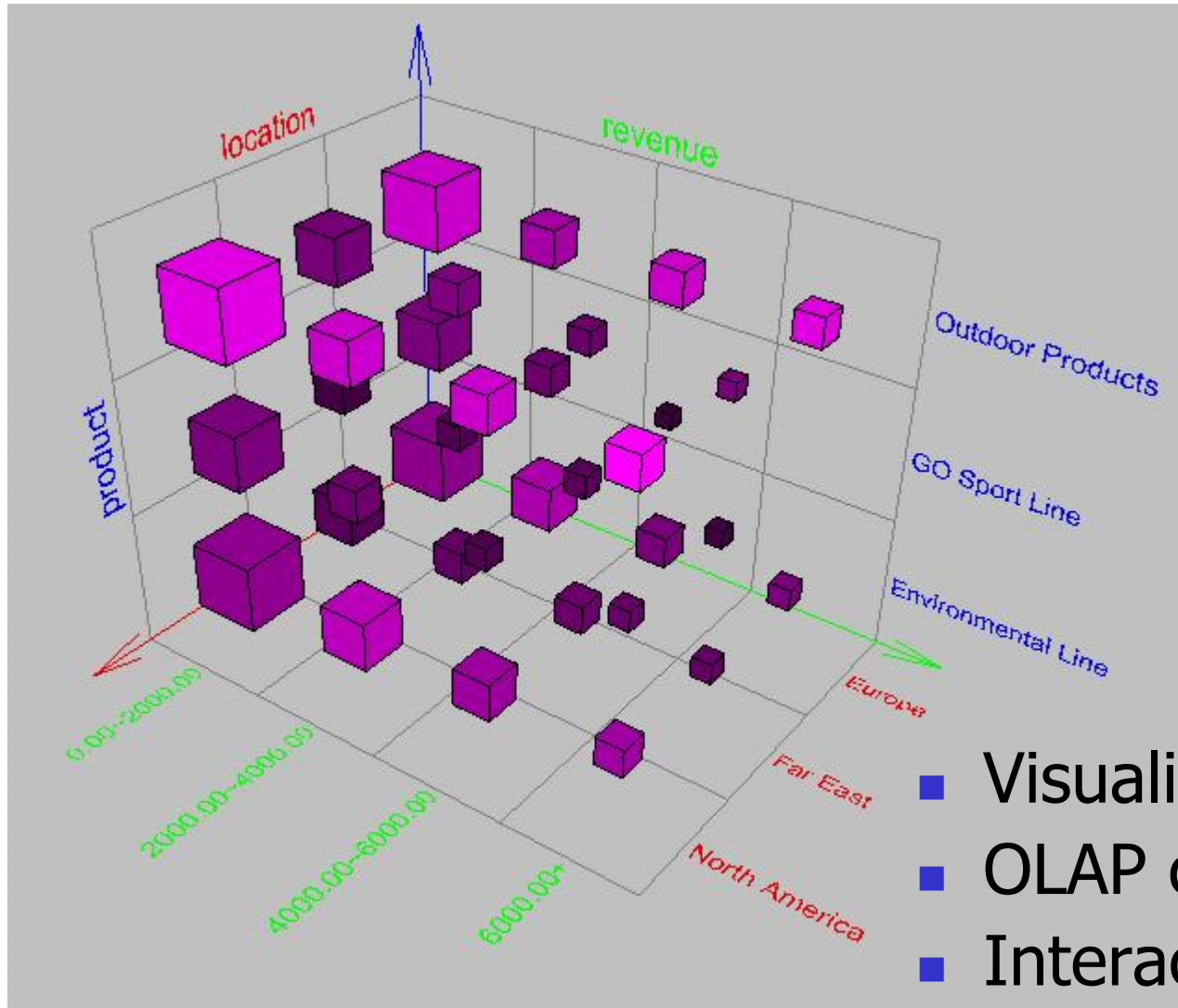
Dimensions: Product, Location, Time
Hierarchical summarization paths



An Extended Data Cube



Browsing a Data Cube



- Visualization
- OLAP capabilities
- Interactive manipulation



Typical OLAP Operations

- **Roll up (drill-up):** summarize data
 - *by climbing up hierarchy or by dimension reduction*
- **Drill down (roll down):** reverse of roll-up
 - *from higher level summary to lower level summary or detailed data, or introducing new dimensions*
- **Slice and dice:** *project and select*
- **Pivot (rotate):**
 - *reorient the cube, visualization, 3D to series of 2D planes*
- **Other operations**
 - ***drill across:** involving (across) more than one fact table*
 - ***drill through:** through the bottom level of the cube to its back-end relational tables (using SQL)*

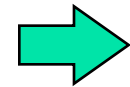
多维分析操作

- 建立在关系聚集操作上的一组复合操作
- 是面向用户的操作
- 切片(slice) & 切块(dice)
- 向上钻取(roll-up), 向下钻取(drill down)
- 旋转(pivoting)

多维分析的基础：聚集

- 基本的分析是求聚集函数(agggregation)
- 例: 求第一天的销售总额
- In SQL: `SELECT sum(amt) FROM SALE WHERE date = 1`

sale	prodId	storeId	date	amt
	p1	c1	1	12
	p2	c1	1	11
	p1	c3	1	50
	p2	c2	1	8
	p1	c1	2	44
	p1	c2	2	4

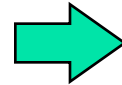


81

多维分析的基础：聚集

- 例：按照每天求销售总额
- In SQL: `SELECT date, sum(amt) FROM SALE GROUP BY date`

sale	prodId	storeId	date	amt
	p1	c1	1	12
	p2	c1	1	11
	p1	c3	1	50
	p2	c2	1	8
	p1	c1	2	44
	p1	c2	2	4

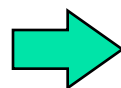


ans	date	sum
	1	81
	2	48

多维分析的基础：聚集

- 例：按照每天和产品求销售总额
- In SQL: `SELECT date, sum(amt) FROM SALE GROUP BY date, prodId`

sale	prodId	storeId	date	amt
	p1	c1	1	12
	p2	c1	1	11
	p1	c3	1	50
	p2	c2	1	8
	p1	c1	2	44
	p1	c2	2	4



ans	prodId	date	amt
	p1	1	62
	p2	1	19
	p1	2	48

—— rollup ——→

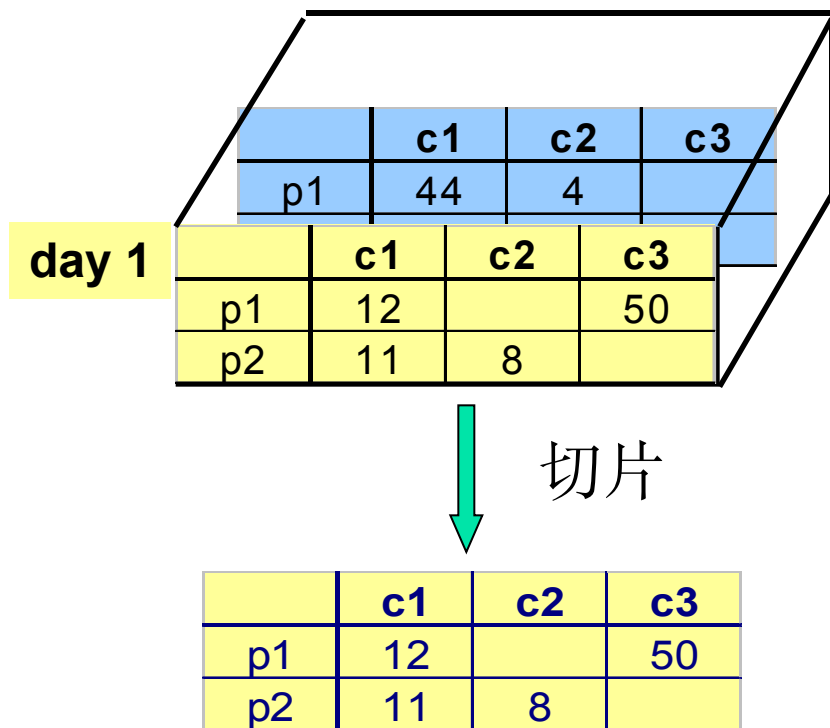
←—— drill-down ——

多维分析的基础：聚集

- 5个标准聚集函数：
sum, count, max, min, ave
- 在group by的基础上还可以使用
“Having” clause
- 可以在维层次上做聚集
 - average by region (within store)
 - maximum by month (within date)

切片/切块操作

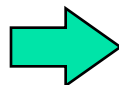
- 实质上对应于where/having 子句



钻取操作

- 钻取操作就是在不同粒度表之间的切换

sale	prodId	storeId	date	amt
	p1	c1	1	12
	p2	c1	1	11
	p1	c3	1	50
	p2	c2	1	8
	p1	c1	2	44
	p1	c2	2	4

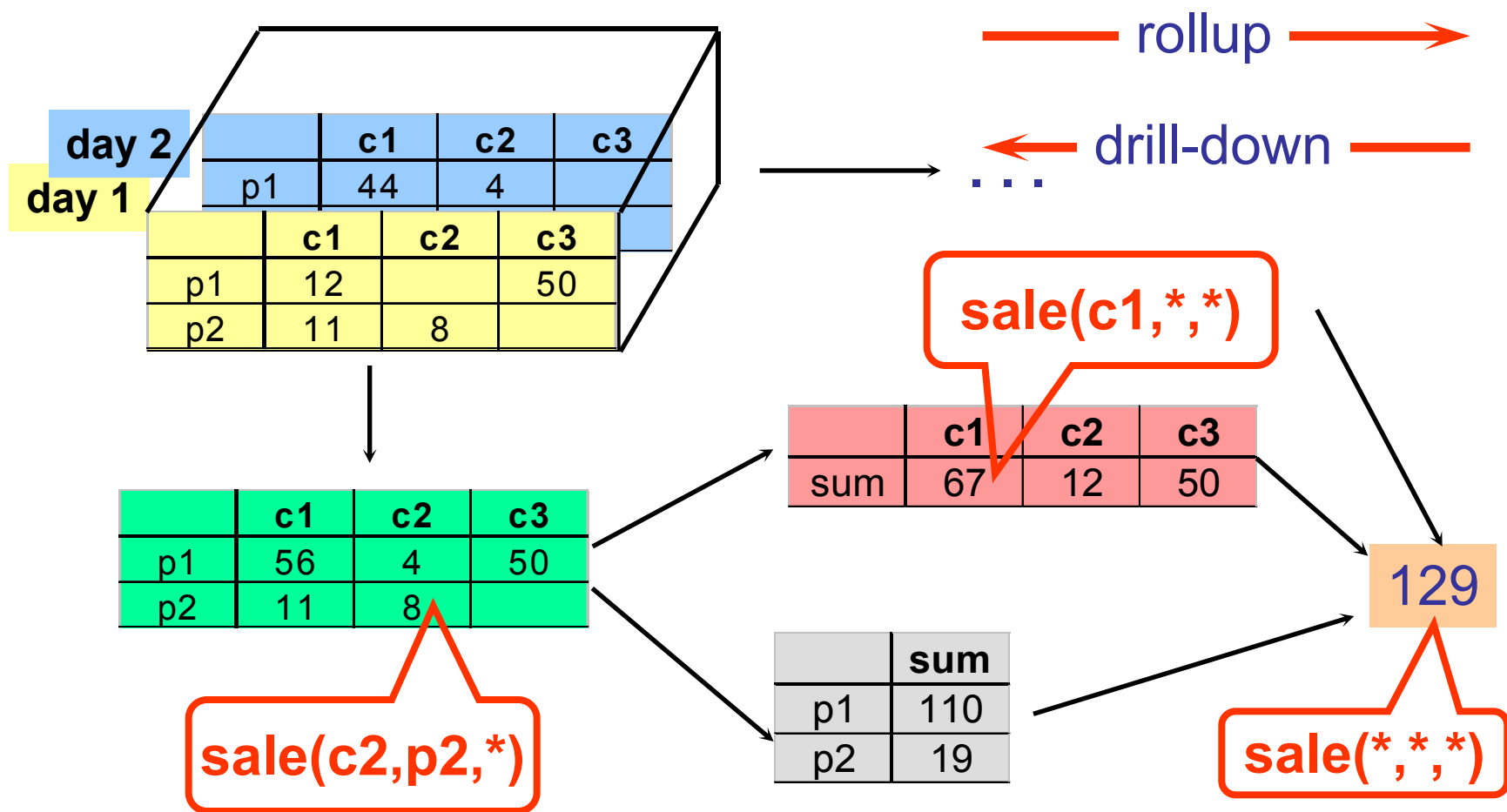


sale	prodId	date	amt
	p1	1	62
	p2	1	19
	p1	2	48

—— rollup —→

← drill-down ——

钻取操作



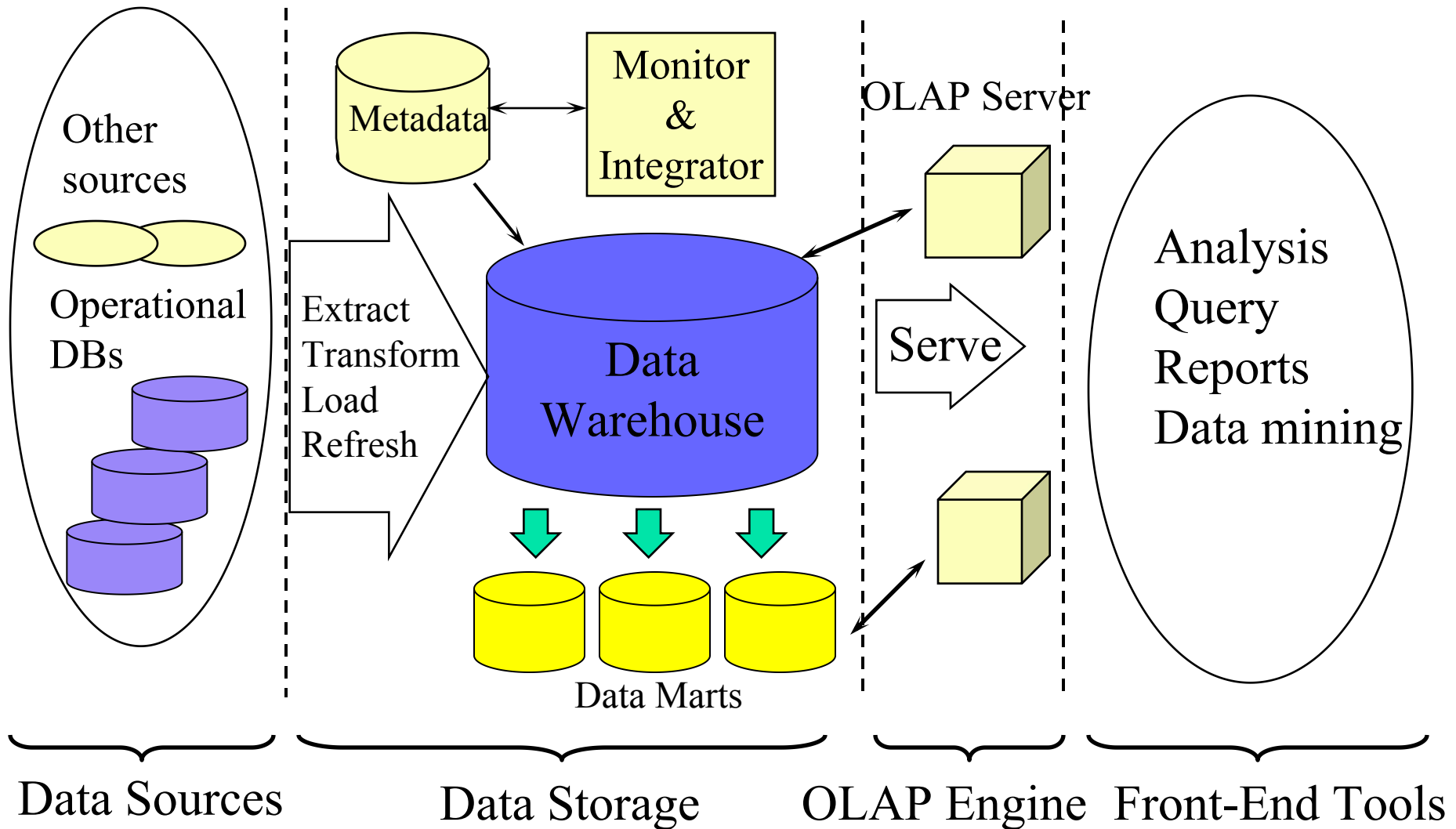
旋转操作



旋转操作



Data Warehouse: A Multi-Tiered Architecture



数据仓库的体系结构

- 数据仓库系统由如下几部分组成：
 - 数据仓库的后台工具
 - 数据仓库服务器
 - **OLAP**服务器
 - 前台工具

数据仓库的体系结构

- 数据仓库的后台工具，包括数据抽取、清洗、转换、装载和维护工具
- 功能：负责把企业或组织中各部门的操作型数据进行转换，装载到数据仓库系统中，同时维护数据仓库和操作型数据源数据的一致性。

数据仓库的体系结构

- 数据仓库服务器一般是关系**DBMS**或扩展的关系**DBMS**。
- **OLAP**服务器透明地为用户提供多维数据，用户不必关心数据到底存储在什么地方。但物理上必须考虑数据存储问题。

Data Warehouse Back-End Tools and Utilities

- Data extraction
 - get data from multiple, heterogeneous, and external sources
- Data cleaning
 - detect errors in the data and rectify them when possible
- Data transformation
 - convert data from legacy or host format to warehouse format
- Load
 - sort, summarize, consolidate, compute views, check integrity, and build indices and partitions
- Refresh
 - propagate the updates from the data sources to the warehouse

OLAP Server Architectures

- Relational OLAP (ROLAP)
 - Use relational or extended-relational DBMS to store and manage warehouse data and OLAP middle ware
 - Include optimization of DBMS backend, implementation of aggregation navigation logic, and additional tools and services
 - Greater scalability
- Multidimensional OLAP (MOLAP)
 - Sparse array-based multidimensional storage engine
 - Fast indexing to pre-computed summarized data
- Hybrid OLAP (HOLAP)
 - Flexibility, e.g., low level: relational, high-level: array
- Specialized SQL servers
 - Specialized support for SQL queries over star/snowflake schemas

ROLAP结构

- 采用关系**DBMS**或扩展的关系**DBMS**来存储和管理数据
- 用**OLAP**服务器做其他剩余的事情，例如聚集计算，查询优化等。
- **Microstrategy**公司的**DSS**服务器和**Infomix**的**Metacube**均采用**ROLAP**结构。
- 各种粒度的数据均以关系表的形式存储在数据仓库中。

MOLAP结构

- 采用多维数组来存储数据
- **Arbor**公司的**Essbase**采用的就是**MOLAP**结构。
- 多维数组存储的优点是可以对数据进行直接定位，不需要索引
- 但在数据稀疏的情况需要进行数据压缩。

HOLAP结构

- 将**ROLAP**和**MOLAP**结合起来，既利用了**ROLAP**可扩展性好的优点，也利用了**MOLAP**计算速度快的优点。
- 例如，可以将细节数据存在关系数据库中，而将综合数据存在**MOLAP**服务器中。
- 微软的**SQL Server 7.0**采用的就是这种结构。

特殊SQL服务器

- 为了满足日益增长的**OLAP**处理的需求，一些关系数据库或数据仓库厂商（如**IBM Red Brick**）对原来的系统进行了扩展，为只读环境下在星星模型基础上进行**SQL**查询提供支持。

Metadata Repository

- Meta data is the data defining warehouse objects. It stores:
- Description of the structure of the warehouse
 - schema, view, dimensions, hierarchies, derived data defn, data mart locations and contents
- Operational meta-data
 - data lineage (history of migrated data and transformation path), currency of data (active, archived, or purged), monitoring information (warehouse usage statistics, error reports, audit trails)
- The algorithms used for summarization
- The mapping from operational environment to the data warehouse
- Data related to system performance
 - warehouse schema, view and derived data definitions
- Business data
 - business terms and definitions, ownership of data, charging policies

Data Warehousing and OLAP Technology: An Overview

- A multi-dimensional data model
- Data warehouse design

Design of Data Warehouse: A Business Analysis Framework

- Four views regarding the design of a data warehouse
 - **Top-down view**
 - allows selection of the relevant information necessary for the data warehouse
 - **Data source view**
 - exposes the information being captured, stored, and managed by operational systems
 - **Data warehouse view**
 - consists of fact tables and dimension tables
 - **Business query view**
 - sees the perspectives of data in the warehouse from the view of end-user

Data Warehouse Design Process

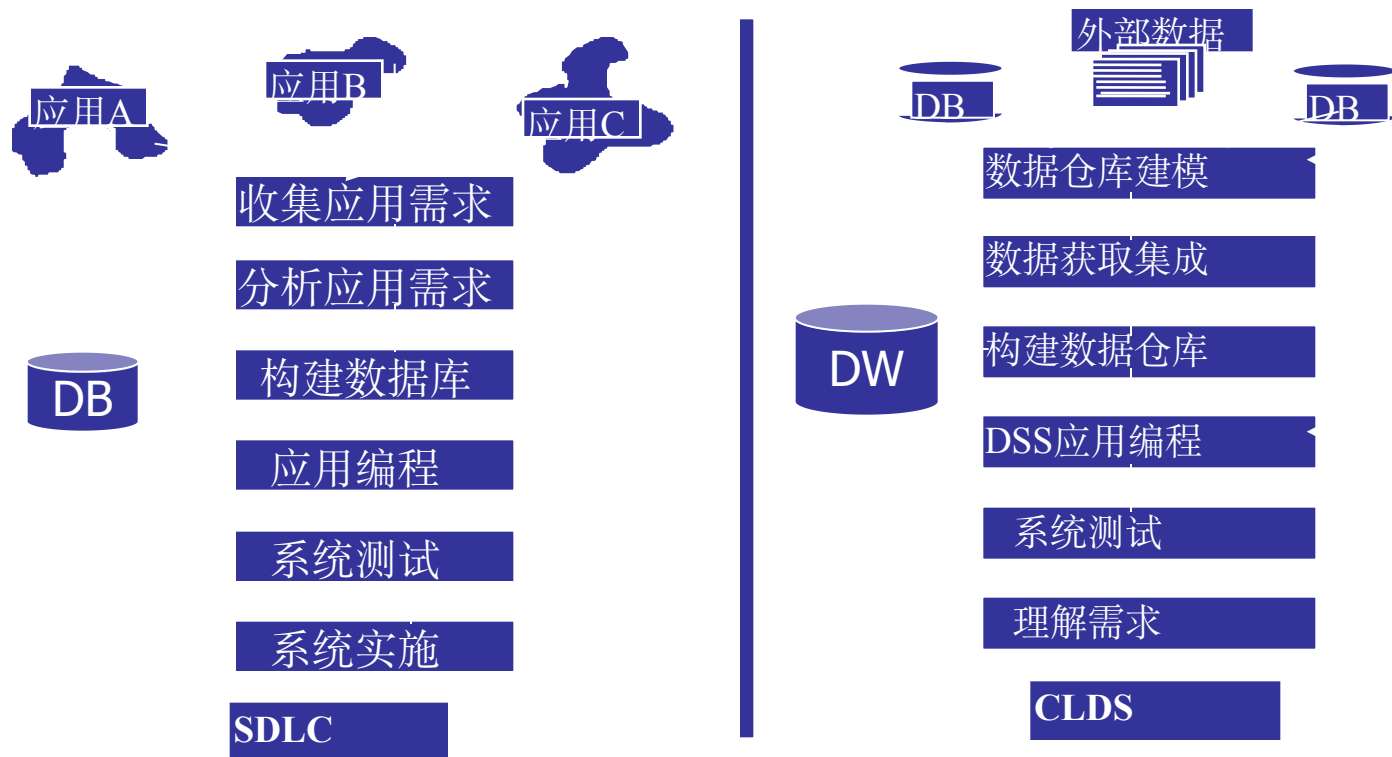
- Top-down, bottom-up approaches or a combination of both
 - Top-down: Starts with overall design and planning (mature)
 - Bottom-up: Starts with experiments and prototypes (rapid)
- From software engineering point of view
 - Waterfall: structured and systematic analysis at each step before proceeding to the next
 - Spiral: rapid generation of increasingly functional systems, short turn around time, quick turn around
- Typical data warehouse design process
 - Choose a **business process** to model, e.g., orders, invoices, etc.
 - Choose the ***grain (atomic level of data)*** of the business process
 - Choose the **dimensions** that will apply to each fact table record
 - Choose the **measure** that will populate each fact table record

数据仓库的设计方法

操作型环境系统的设计一般采取系统生命周期法：
(**SDLC – Systems Development Life Cycle**)

数据仓库的设计方法：**CLDS**方法(与**SDLC**相反)

SDLC与CLDS方法比较



数据仓库设计的三级数据模型

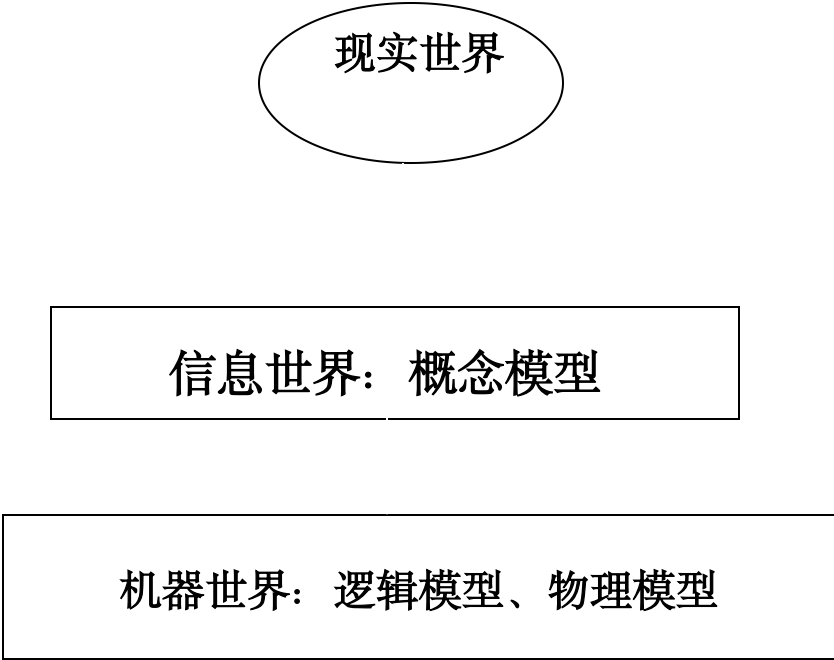
数据仓库的三级数据模型与数据库的区别:

- 1 数据仓库的数据模型中不包含纯操作型的数据
- 2 数据仓库的数据模型扩充了码结构, 增加了时间属性作为 码的一部分
- 3 数据仓库的数据模型中增加了一些导出数据

数据仓库设计的三级数据模型

- 上述三点差别也就是操作型环境中的数据与数据仓库中的数据之间的差别,同样是数据仓库为面向数据分析处理所要求的。
- 虽然存在着这样的差别,在数据仓库设计中,仍然存在着三级数据模型,即:
 - 概念模型
 - 逻辑模型
 - 物理模型

概念模型



现实世界

信息世界：概念模型

机器世界：逻辑模型、物理模型

概念模型

概念模型常用的表示方法:

E - R 方法 (实体 - 联系方法) 描述工具: E—R 图

1. 长方形表示实体, 数据仓库中表示主题, 框内写上主题名

2. 椭圆形表示主题的属性

无向边把主题与其属性连接起来

3. 菱形表示主题之间的联系, 菱形框内写上联系的名字

无向边把菱形分别与有关的主题连接

无向边旁标上联系的类型

若联系也有属性 用无向边连接属性和菱形

逻辑模型

在RDBMS中：关系模型

主题以及主题之间的联系都用关系来表示

关系模型的基本概念：

关系 一个二维表；

元组 表中的一行称为一个元组；

属性 表中的一列称为属性，给每一列起一个名称即属性名

主码 表中的某个属性组，它们的值唯一地标识一个元组

域 属性的取值范围

分量 元组中的一个属性组

关系模式 对关系的描述

关系名 (属性名1, 属性名2, ..., 属性名n)

物理模型

数据仓库的物理模型：是逻辑模型在数据仓库中的实现，如：

- 物理存取方式
- 数据存储结构
- 数据存放位置
- 存储分配等等

物理模型

物理模型设计考虑的主要因素：

I/O存取时间

空间利用率

维护代价

物理模型

- 物理模型设计时需要考虑如何提高性能:
 - 数据量很大, 涉及大规模数据的查询
 - 主要是要提高系统的I/O性能
 - 应尽量减少每次查询处理要求的I/O次数
 - 使每次I/O能返回尽量多的记录。

物理模型

由于数据仓库的数据极少甚至不再更新，可采取如下技术来提高数据仓库的性能：

- 1** 合并表
- 2** 建立数据序列
- 3** 引入冗余
- 4** 进一步细分数据
- 5** 生成导出数据
- 6** 建立广义索引
- 7** 粒度划分
- 8** 分割

1. 合并表

在数据仓库中，往往存在一些例行的分析处理，它们要求的查询也是例行的、相对固定的。

当某一例行的查询涉及固定的多个表的数据项，那么就需要首先对这几个表进行连接操作，如果这几个表的记录分散存放在几个物理块中时，多个表的存取和连接操作的代价会很大。

为了节省I/O开销，可以把这些表的记录混合存放在一起，就可以降低表连接操作的代价。

1. 合并表

学号#s	学生姓名SN	所属系名SD	学生年龄SA
S1	A	CS	20
S2	B	CS	21
S3	C	MA	19
S4	D	CI	19
S5	E	MA	20

学生选课表SC

合并

学号	课程号	学习成绩
S1	C1	A
S1	C2	B
S1	C3	C
S1	C5	C
S2	C1	A
S2	C2	A
S2	C4	A
S3	C2	B
S3	C4	D
S4	C3	C
S4	C5	B
S5	C1	B
S5	C3	A
S5	C4	A

合并表

S1ACS20	S1C1A	S1C2B	S1C3C	S1C5C
S2BCS21	S2C1A	S2C2A		
S3C4D	S4DCI19	S4C3A		
				

SC表记录

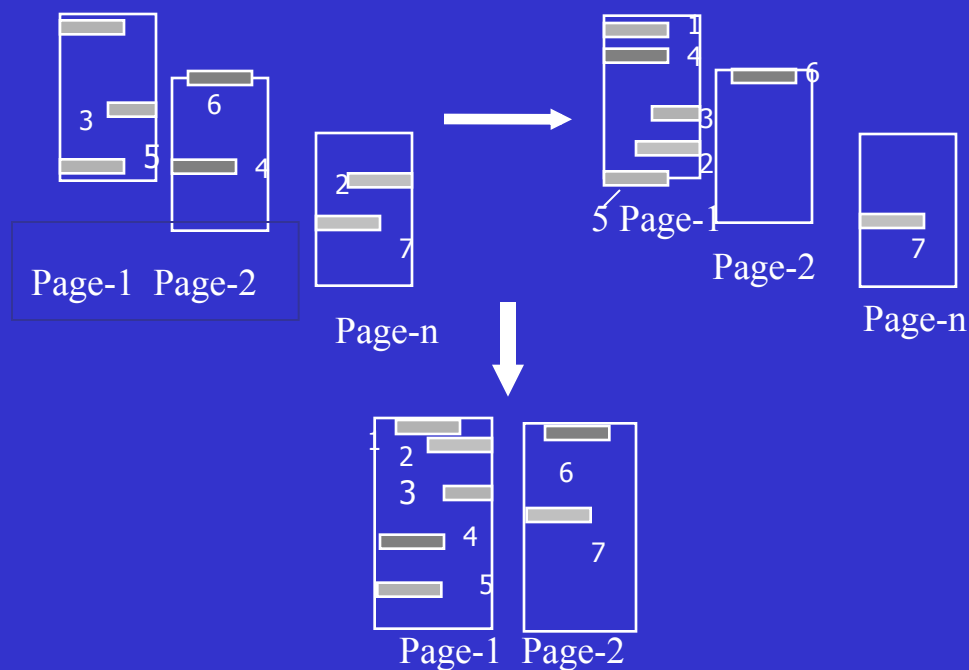
S表记录

2. 建立数据序列

重复调页

页面1(记录1)---页面n(记录2)---页面1(记录3)---页面2(记录4)---

1



-
- 数据序列：将数据严格按照处理顺序存放在一个或多个连续的物理块中，这样就可以在同一次调页中处理更多的记录，将物理I/O次数降到最低。

3. 引入冗余

数据分析处理通常要涉及不同表的多个属性,一些表的某些属性可能在许多地方都要用到

如果这些属性上的值不常更新,则将这些属性复制到多个主题中,从而减少处理时被存取的表的个数。

3. 引入冗余

例：在商场DW中，在“商品”主题中有一个保存商品固定信息的关系表：
商品(商品号、商品名、类别、...),
而在商品销售表或采购表中则只存商品号。

但几乎没有人问：

“×××号商品的销售情况如何？”

而经常问：

“某品牌的冰箱的销售可好？” 或 “今年什么类型的洗衣机最畅销？”

这类问题以商品的一些具体描述信息作为分析的限定条件，且涉及的有关销售的数据量又很大，这样就不得不反复存取商品表，与大量的销售表记录进行连接或半连接操作。

如果将商品表的一些特定属性加入到销售表或采购表中，即增加数据冗余，就可以省去这一步连接操作，减少访问的代价。

4. 表的物理分割

逻辑分割：将一个表按照一定的分割标准划分成两个或多个表。

物理分割：根据表中各属性的存取频率分成两个或多个表，将具有相似访问频率的属性存放在一起，使得每次访问的效率较高。

5. 生成导出数据

导出数据：原始数据上进行汇总或计算的结果
预处理，提高联机分析速度

更高级别的公用数据，免去不同用户重复计算

6. 建立广义索引

广义索引:

用来记录具有某些特殊性质数据的索引
比如最值, top-k值等

特点: 非常小, 大大提高查询效率

例: 记录当月销售最糟糕的10种商品的名称的索引

6. 建立广义索引

建立 “广义索引”：

抽取数据并向数据仓库中装载时建立

每次数据仓库装载时，重新生成“广义索引”的内容

不要为了建立“广义索引”而去扫描数据仓库

管理 “广义索引”：

元数据中完整地定义“广义索引”说明

7. 粒度划分

粒度划分的决定性因素：

总的行数不是总的数据量

划分粒度步骤：

1. 估算数据的行数和所需的DASD数
(direct access storage device) ；
2. 根据估算出的行数和DASD，决定是否要划分粒度，如何划分粒度

7. 粒度划分

对每一已知表：

 计算一行所占字节数的

 最大值

 最小值

对一年内：

 统计可能出现的数据行数的

 最大行数、

 最小行数

对五年内：

 统计可能出现的数据行数的

 最大行数、

 最小行数

7. 粒度划分

计算每个表的码所占的字节数:

一年产生的数据可能占用的最大空间 = 最大值 × 一年内最大行数 + 索引空间

一年产生的数据可能占用的最小空间 = 最小值 × 一年内最小行数 + 索引空间

五年产生的数据可能占用的最大空间 = 最大值 × 五年内最大行数 + 索引空间

五年产生的数据可能占用的最小空间 = 最小值 × 五年内最小行数 + 索引空间

直到计算完所有表

7. 粒度划分

2、根据估算出的数据行 和DASD，决定是否要划分粒度，如何划分粒度

第一年： 10万行左右，加一个综合级别

1,00万行左右，采用多重粒度

五年内： 1,00万行，划分粒度

> 10,00万行，多重粒度

8 数据分割

数据分割: 物理存储单元 --- 分片
逻辑上整体的数据
分割成较小的、可以独立管理

8 数据分割

数据仓库中的数据分片方式:

1. 水平分片: 按一定的条件将一个关系按行分为若干不相交的子集

2. 垂直分片: 将关系按列分为若干子集

3. 导出分片 (导出水平分片) :

水平分片的条件不是本身属性的条件, 而是其他关系的属性的条件

例如: SC (Sno, Cno, Grade)

不是按照学号或课程号或成绩的某类条件分片

而是按照学生年龄 > 18 岁和 ≤ 18 岁来分片

(学生年龄是学生关系Student的属性)

8 数据分割

4.混合分片：按上述三种分片方式得到的片段继续按另一种方式分片

1		
2		
3-1	3-2	3-3

(a)

1-1	2
1-2	

(b)

混合分片

8 数据分割

数据分割优点:

1 提高系统性能

“1996年家电类商品销售的季节分布如何？”

“每年的第一季度商品销售在各类商品上的分布情况是怎样的？”

2 提高灵活性

修改数据定义容易

不同年份的险种的定义描述是相互独立

8 数据分割

选择分割标准考虑的因素：

- 数据量（而非记录行数）

数据量较小，不分割

数据量很大，考虑采用多重标准的组合

- 数据分析处理的要求

“商品”主题：按商品大类，商品小类和时间标准分割

“供应商”主题：按地理位置（供应商的地址），时间分割

- 简单易行

易于实施，按时间，按业务类型

- 粒度划分策略等

将数据分割标准与粒度的划分策略统一考虑

“商品”主题：

关于商品销售数据的粒度是按时间和商品
类别综合

每一粒度层次上：

可以按时间和商品类别的组合进行分割

数据仓库设计步骤

- 1 不能采用同传统的OLTP数据库一样的设计方法
- 2 数据仓库系统的原始需求不明确，且不断变化与增加，开发者最初并不能确切了解到用户的明确而详细的需求
- 3 用户所能提供的是需求的大方向以及部分需求，不能较准确地预见到以后的需求
- 4 采用原型法是比较合适的

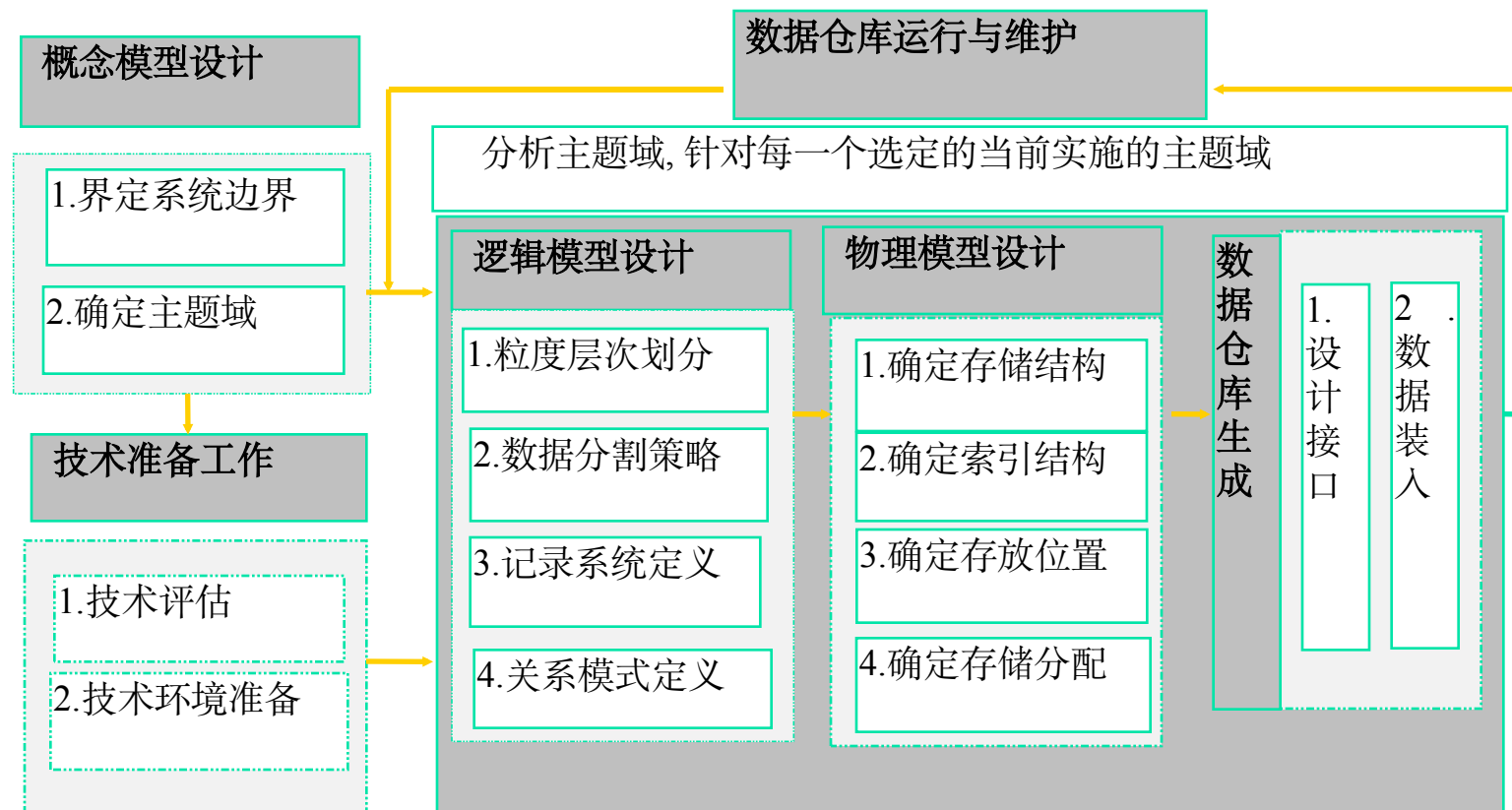
原型法的思想:

从构建系统的简单的基本框架着手，不断丰富与完善整个系统——
是不断循环、反馈而使系统不断增长与完善的过程

自始至终要求决策人员和开发者的共同参与和密切协作

数据仓库设计的大体步骤:

- 1.概念模型设计
- 2.技术准备工作
- 3.逻辑模型设计
- 4.物理模型设计
- 5.数据仓库生成
- 6.数据仓库运行与维护



References (I)

- S. Agarwal, R. Agrawal, P. M. Deshpande, A. Gupta, J. F. Naughton, R. Ramakrishnan, and S. Sarawagi. On the computation of multidimensional aggregates. VLDB'96
- D. Agrawal, A. E. Abbadi, A. Singh, and T. Yurek. Efficient view maintenance in data warehouses. SIGMOD'97.
- R. Agrawal, A. Gupta, and S. Sarawagi. Modeling multidimensional databases. ICDE'97
- K. Beyer and R. Ramakrishnan. Bottom-Up Computation of Sparse and Iceberg CUBEs.. SIGMOD'99.
- S. Chaudhuri and U. Dayal. An overview of data warehousing and OLAP technology. ACM SIGMOD Record, 26:65-74, 1997.
- OLAP council. MDAPI specification version 2.0. In <http://www.olapcouncil.org/research/apily.htm>, 1998.
- Y. Chen, G. Dong, J. Han, B. W. Wah, and J. Wang, " Multi-Dimensional Regression Analysis of Time-Series Data Streams ", VLDB'02.
- G. Dong, J. Han, J. Lam, J. Pei, K. Wang. Mining Multi-dimensional Constrained Gradients in Data Cubes. VLDB' 01

References (II)

- J. Gray, S. Chaudhuri, A. Bosworth, A. Layman, D. Reichart, M. Venkatrao, F. Pellow, and H. Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab and sub-totals. *Data Mining and Knowledge Discovery*, 1:29-54, 1997.
- J. Han. [Towards on-line analytical mining in large databases](#). *ACM SIGMOD Record*, 27:97-107, 1998.
- J. Han, Y. Cai and N. Cercone, [Knowledge Discovery in Databases: An Attribute-Oriented Approach in \(VLDB'92\)](#), Vancouver, Canada, August 1992, pp. 547-559.
- J. Han, J. Pei, G. Dong, K. Wang. Efficient Computation of Iceberg Cubes With Complex Measures. *SIGMOD'01*
- V. Harinarayan, A. Rajaraman, and J. D. Ullman. Implementing data cubes efficiently. *SIGMOD'96*
- L. V. S. Lakshmanan, J. Pei, and J. Han, [Quotient Cube: How to Summarize the Semantics of a Data Cube](#), Proc. 2002 Int. Conf. on Very Large Data Bases (VLDB'02), Hong Kong, China, Aug. 2002.
- Microsoft. OLEDB for OLAP programmer's reference version 1.0. In <http://www.microsoft.com/data/oledb/olap>, 1998.
- K. Ross and D. Srivastava. Fast computation of sparse datacubes. *VLDB'97*.
- K. A. Ross, D. Srivastava, and D. Chatziantoniou. Complex aggregation at multiple granularities. *EDBT'98*.

References (III)

- S. Sarawagi, R. Agrawal, and N. Megiddo. Discovery-driven exploration of OLAP data cubes. EDBT'98.
- G. Sathe and S. Sarawagi. Intelligent Rollups in Multidimensional OLAP Data. In *Proc. Int. Conf. of Very Large Data Bases (VLDB'01)*, Rome, Italy, pp. 531-540
- E. Thomsen. OLAP Solutions: Building Multidimensional Information Systems. John Wiley & Sons, 1997.
- D. Xin, J. Han, X. Li, B. W. Wah, "Star-Cubing: Computing Iceberg Cubes by Top-Down and Bottom-Up Integration", Proc. 2003 Int. Conf. on Very Large Data Bases (VLDB'03), Berlin, Germany, Sept. 2003.
- W. Wang, H. Lu, J. Feng, J. X. Yu, Condensed Cube: An Effective Approach to Reducing Data Cube Size. ICDE'02.
- Y. Zhao, P. M. Deshpande, and J. F. Naughton. An array-based algorithm for simultaneous multidimensional aggregates. SIGMOD'97.